Pixelle-Video 从入门到精通

项目	说明
主题	Pixelle-Video（AI 全自动短视频引擎）
适用版本	基于主分支最新代码：最新正式发布为 v0.1.15（2026 年 1 月 27 日）；数字人口播、图生视频、动作迁移等模块已随该版本提供；直连 API 媒体模型配置为 2026 年 6 月主分支新增，详见第 10 章版本说明
适用系统	Windows、macOS、Linux（含 Docker 部署）
资料来源与核实日期	2026 年 10 月 30 日，依据官方 README、官方文档站（aidc-ai.github.io/Pixelle-Video/zh）、官方发布说明与仓库配置文件逐项核对
预设读者	具备基本计算机操作能力的视频创作者；第一、二部不要求编程经验，第三部分定制章节需要基础的命令行与 Python 知识
学习成果	读者能够独立完成从安装配置到生成第一条视频的全流程，并按需进行视觉风格、声音、模板的定制，最终掌握工作流定制、直连模型调用、扩展流水线与自动化批量生产

本教程为教学性文档，非官方出版物。文中命令、界面名称与参数截至核实日均与官方资料一致；未能在官方资料中获得准确依据的细节一律未写入。各服务商的计费标准变动频繁，涉及费用的内容仅说明结构，不给出具体数额，以各平台现行价格为准。

目录

- 第一部 入门篇：从零生成第一条视频
  - 第 1 章 认识 Pixelle-Video：定位、能力全景与生成流程
  - 第 2 章 安装部署：Windows 整合包与源码安装
  - 第 3 章 系统配置：LLM、ComfyUI、RunningHub 与直连 API
  - 第 4 章 生成第一个视频：三栏 Web 界面完整走读
- 第二部 进阶篇：打磨内容与视听风格
  - 第 5 章 内容创作：AI 文案、固定文案与自定义素材
  - 第 6 章 语音体系：TTS 工作流、试听预览与声音克隆
  - 第 7 章 视觉体系：图像工作流、尺寸与提示词前缀
  - 第 8 章 视频模板：模板体系与画面版式控制
- 第三部 高级篇：定制、扩展与自动化
  - 第 9 章 ComfyUI 本地部署与工作流定制
  - 第 10 章 直连 API 媒体模型与 API 视频生成
  - 第 11 章 扩展流水线：数字人口播、图生视频、动作迁移
  - 第 12 章 精通实战：批量生成、历史记录、性能优化与故障排查
- 附录 A 常见错误速查表
- 附录 B 术语表

第一部 入门篇：从零生成第一条视频

第 1 章 认识 Pixelle-Video：定位、能力全景与生成流程

一条短视频的传统生产流程要跨越文案撰写、配图设计、配音录音、字幕排版和最终合成多个环节，任何一环都需要专门的工具与技能。Pixelle-Video 的做法是把上述环节编排成一条自动化流水线：使用者输入一个主题，系统自动撰写文案、为每个画面生成配图或视频、合成解说语音、叠加背景音乐，最后输出可直接发布的成片。本章先建立对该项目的整体认识，包括它能做什么、内部流程如何运转、部署与付费路径有哪些选择。

1.1 项目定位与许可证

Pixelle-Video 是一个开源的 AI 全自动短视频生成引擎，采用 Apache 2.0 许可证发布。官方 README 中给出的代码仓库链接为 AIDC-AI/Pixelle-Video；截至 2026 年 10 月 30 日核实，GitHub 上的 ATH-MaaS/Pixelle-Video 提供同一项目内容，本教程的克隆命令使用后者地址。官方文档站为 aidc-ai.github.io/Pixelle-Video/zh，Windows 一键整合包发布在项目 Releases 页面。

1.2 能力全景

Pixelle-Video 的核心能力可以概括为一句话：输入一个主题或一份现成文案，其余环节由一条可配置的流水线自动完成。下表列出官方 README 中明确支持的能力项。

能力项	说明
AI 文案生成	根据主题自动创作解说词，无需人工写稿
AI 配图生成	为每个分镜自动生成插图，风格可配置
AI 视频生成	使用 WAN 2.1 等视频模型生成动态片段
直连模型 API	可不经过 ComfyUI，直接调用 DashScope、OpenAI、Seedream、Seedance、Kling 等图像与视频服务
AI 语音合成	支持 Edge-TTS、Index-TTS 等多种 TTS 方案，部分方案支持声音克隆
背景音乐	内置音乐可选，也支持放入自定义音乐文件
视觉模板	多种 HTML 模板控制画面版式，覆盖竖屏、横屏、方形
文案模型接入	文案生成支持 GPT、通义千问、DeepSeek、Ollama 等
扩展流水线	数字人口播、图生视频、动作迁移、自定义素材分析

1.3 核心概念速览

后续章节会反复使用以下术语。此处先给出简明定义，详细用法在对应章节展开。

术语	英文	定义
分镜	Scene	视频的基本组成单元。通常一句或一段解说对应一个分镜，每个分镜包含画面与对应配音
大语言模型	LLM (Large Language Model)	负责把主题扩写成结构化文案的文本模型，如通义千问、GPT 系列
文本转语音	TTS (Text-to-Speech)	把解说文本合成为人声语音的技术
ComfyUI	—	基于节点连线方式的生成式 AI 图形编排环境，Pixelle-Video 通过它运行图像、视频与语音生成模型
工作流	Workflow	描述一次生成任务的 JSON 文件，包含模型、节点与参数；本项目按 `image_`、`video_`、`tts_` 前缀区分用途
RunningHub	—	提供云端 ComfyUI 运算设备（24GB 或 48GB 显存机型）的托管平台，本项目通过它的 API 调用云端工作流
直连 API	Direct API	绕过 ComfyUI，直接调用模型服务商官方接口的方式
模板	Template	以 HTML 与 CSS 编写的单分镜画面版式文件，决定文字与背景的排版风格
背景音乐	BGM (Background Music)	叠加在人声解说之下的音乐音轨

1.4 生成流程全貌

整体流程按官方说明分为四步：文案生成、配图规划、逐帧处理、视频合成。理解每一步的职责，有助于后续定位问题出在哪个环节。

1. **文案生成**：将主题交给 LLM，产出带分镜结构的解说稿；
2. **配图规划**：为每个分镜生成画面描述，交由图像或视频生成服务出图；
3. **逐帧处理**：选定模板把每个分镜渲染为画面，并完成语音合成；
4. **视频合成**：按解说时长串联全部分镜，混入 BGM 与解说，输出最终视频文件到 `output/` 目录。

四个环节均支持替换模型或方案：文案可换 LLM，配图与动态画面可在 ComfyUI 工作流、RunningHub 云端工作流、直连 API 三类途径之间选择，语音可在多种 TTS 工作流之间选择。

1.5 部署路径与付费结构

部署方式与媒体生成途径是两件独立的事，组合起来决定了使用成本。下表为三种部署方式的概况，安装细节见第 2 章。

部署方式	适用人群	要点
Windows 一键整合包	Windows 用户	无需安装 Python、uv 或 ffmpeg，解压后双击 start.bat 运行
源码安装	macOS 与 Linux 用户、需要定制的开发者	需 Python 3.10 以上、uv 包管理器与 ffmpeg
Docker	服务器或长期运行场景	以 docker-compose 一键启动 Web 与 API 两个服务

媒体生成途径的选择逻辑如下，费用结构以官方 README 的说明为准。

- **完全免费方案**（官方推荐给本地有显卡的使用者）：LLM 使用本机 Ollama，配图使用本机部署的 ComfyUI，API 侧零消耗；
- **推荐方案（性价比）**：LLM 使用通义千问（按调用量计费，成本较低），配图仍用本机 ComfyUI；
- **云端方案**：LLM 使用 OpenAI，配图使用 RunningHub 云端工作流，优点是不需要本地显卡，缺点是按量付费且费用较高。

除 LLM 与媒体生成外，TTS 若使用 Edge-TTS 则不产生费用；RunningHub 部分工作流按其平台计费规则消耗积分。各途径的具体配置在本教程第 3 章集中说明。

第 2 章 安装部署：Windows 整合包与源码安装

本章解决从零到可运行的问题。三种部署方式中，Windows 使用者优先选择整合包，它免除了全部环境搭建；macOS 与 Linux 使用者选择源码安装；需要在服务器长期运行或不便安装本地环境的场景选择 Docker。完成任意一种方式后，浏览器中应当能看到 http://localhost:8501 的 Web 界面。

2.1 Windows 一键整合包

整合包面向不维护 Python 环境的使用者，设备要求为可正常使用的 Windows 系统；若后续要用本地 ComfyUI 生成图像，需要 NVIDIA 6GB 以上显存的显卡，该要求来自官方系统需求说明。

安装步骤如下。

1. 访问项目 Releases 页面（github.com/ATH-MaaS/Pixelle-Video/releases/latest），下载最新的 Windows 一键整合包压缩文件；截至核实日，最新版本为 v0.1.15；
2. 将压缩包解压到任意非系统关键目录；
3. 双击解压目录中的 `start.bat`，等待服务初始化完成；
4. 浏览器自动打开 http://localhost:8501，即为 Web 界面入口。

整合包已内置 Python、uv、ffmpeg 与全部依赖，首次使用只需按第 3 章配置 API 密钥即可开始生成。停止使用时关闭命令行窗口即可；再次使用时重复第 3 步。

2.2 源码安装

前置依赖

源码安装需要三样东西：Git、Python 包管理器 uv、音视频处理工具 ffmpeg。Python 3.10 或更高版本也是必需条件。

uv 是一种快速、可靠的 Python 包与项目管理工具，随后用它来安装依赖并启动程序。macOS 与 Linux 在终端执行官方安装脚本：

```
$ curl -LsSf https://astral.sh/uv/install.sh | sh
```

Windows 使用者在 PowerShell 执行（命令以 uv 官方文档为准）：

```
$ powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

安装完成后运行以下命令验证，正常情况输出 uv 的版本号。

```
$ uv --version
```

ffmpeg 负责最终的音频与视频合成，是必装项。各系统安装方式如下，示例均为命令行安装方式。

系统	安装命令
macOS	`brew install ffmpeg`
Ubuntu / Debian	`sudo apt update`，然后 `sudo apt install ffmpeg`
Windows	从 ffmpeg.org/download.html 下载，解压后将其 `bin` 目录加入系统 PATH 环境变量

安装完成后验证：

```
$ ffmpeg -version
```

获取代码与安装依赖

```
$ git clone https://github.com/ATH-MaaS/Pixelle-Video.git
$ cd Pixelle-Video
$ uv sync
```

`uv sync` 会自动创建虚拟环境并按锁定文件安装全部依赖。

若偏好使用传统 pip，官方文档也提供了等价做法：先创建并激活虚拟环境，再以可编辑模式安装本包。

```
$ python -m venv venv
```

激活虚拟环境，Windows 执行 `venv\Scripts\activate`；macOS 与 Linux 执行 `source venv/bin/activate`。随后安装：

```
$ pip install -e .
```

启动与验证

```
$ uv run streamlit run web/app.py
```

该命令使用 uv 找到虚拟环境并以 Streamlit（一种用于构建数据与 AI 应用 Web 界面的 Python 框架）启动 Web 界面。命令执行后，浏览器自动打开 http://localhost:8501。若改用 pip 并已激活虚拟环境，等价命令为 `streamlit run web/app.py`。

2.3 Docker 部署

以下内容依据仓库中的 docker-compose.yml 与官方文档核实。Docker 方案适合服务器长期运行或希望不污染本机 Python 环境的场景，同一套配置会同时启动两个服务：API 后端（端口 8000）与 Web 界面（端口 8501）。

前置条件为已安装 Docker 与 Docker Compose。进入克隆好的项目目录后执行：

```
$ docker-compose up -d
```

中国大陆网络环境下可改为使用清华镜像源加速构建：

```
$ USE_CN_MIRROR=true docker-compose up -d
```

执行时会发生三件事，理解它们有助于后续排错。

1. 名为 init 的初始化容器先运行：若项目根目录不存在 `config.yaml`，会自动从 `config.example.yaml` 复制一份；若发现 `config.yaml` 被错误挂载成了目录，会自动删除该目录再重建文件；
2. `pixelle-video-api` 容器在 8000 端口提供 REST API，健康检查地址为 `/health`；
3. `pixelle-video-web` 容器在 8501 端口提供 Web 界面，即使用者日常操作的入口。

持久化与自定义资源的位置需要注意：`config.yaml`、`data/`、`output/` 三个路径以卷方式挂载在宿主目录。自定义素材（音乐、模板、工作流）在 Docker 部署中放入 `data/` 下对应子目录（如 `data/bgm`、`data/templates`、`data/workflows`），运行时会覆盖镜像内的内置默认资源；成片输出在 `output/`。

2.4 项目目录导览

无论哪种部署方式，以下目录承担固定职责。掌握它们，后续章节修改资源时才能准确落点。

路径	职责
`web/`	Streamlit Web 界面前端代码
`api/`	REST API 后端代码
`workflows/`	图像、视频与 TTS 工作流 JSON 文件，分 `selfhost/` 与 `runninghub/` 两组
`templates/`	视频模板 HTML 文件，按视频尺寸目录分组
`bgm/`	内置背景音乐，可放入自定义音乐文件
`output/`	生成的成片输出目录
`config.example.yaml`	配置文件模板，含全部配置项与注释
`config.yaml`	实际生效的配置文件，保存配置时自动生成

第 3 章 系统配置：LLM、ComfyUI、RunningHub 与直连 API

安装完成后程序仍处于不可生成状态：文案需要 LLM 服务，画面需要媒体生成服务，二者缺一不可。本章把首次配置一次讲清，配置完成后即可进入第 4 章生成第一个视频。配置入口有两个，效果等同：Web 界面侧边栏中展开「系统配置」面板填写并点击「保存配置」；或编辑项目根目录的 `config.yaml` 文件。`config.yaml` 在首次保存配置时自动生成，也可以从 `config.example.yaml` 手动复制后修改，该示例文件顶部带有明确警告：切勿将 `config.yaml` 提交到版本库。

3.1 LLM 配置：文案生成服务

LLM 决定文案质量，也是首次配置的第一项。Web 界面提供预设下拉菜单，选择某个预设后 base_url 与 model 两项会自动填充，使用者只需填入 API Key 并通过右侧「获取 API Key」链接前往对应平台注册取密钥。四个官方预设对应的服务地址与模型如下，来源为 config.example.yaml 中的预设注释。

预设	Base URL	Model	密钥来源	成本特点
通义千问	`https://dashscope.aliyuncs.com/compatible-mode/v1`	`qwen-max`	阿里云百炼平台	按量计费，官方定位为性价比方案
GPT-4o	`https://api.openai.com/v1`	`gpt-4o`	OpenAI	按量计费，费用相对较高
DeepSeek	`https://api.deepseek.com`	`deepseek-chat`	DeepSeek 开放平台	按量计费，费用较低
Ollama（本机）	`http://localhost:11434/v1`	`llama3.2`	无需密钥	免费，要求本机已运行 Ollama 并已下载对应模型

任何兼容 OpenAI SDK 接口的服务都可以接入，配置只涉及三个字段：`api_key`（密钥）、`base_url`（服务地址）、`model`（模型名称）。Ollama 属于本机部署的大模型运行框架，选择该预设前需先在本机安装 Ollama 并执行 `ollama pull` 下载所选模型，具体做法见 Ollama 官方文档。

v0.1.15 引入了两项便利功能：配置界面支持从 API 自动获取该密钥可用的模型列表，免去手工查名；同时提供 LLM 连接测试按钮，用于在保存前验证 Key 与地址无误。

3.2 画面生成服务：三种途径总览

画面（图像与视频）生成有三条可选途径，本教程按部署形态归纳为下表；三者可共存，逐个工作流指定使用哪一条途径。

途径	需配置	需本地设备	适用场景
本地 ComfyUI（SelfHost）	ComfyUI 服务地址	NVIDIA 6GB 以上显存显卡	长期免费生成，愿意维护本地环境
RunningHub 云端	RunningHub API Key	无	不装本地环境即可使用
直连模型 API	对应供应商密钥与 Base URL	无	只想调用某家模型的图像与视频服务

3.3 ComfyUI 与 RunningHub 配置

Web 界面的「ComfyUI / RunningHub 配置」区对应 config.yaml 中 `comfyui` 段。

ComfyUI 侧配置两个字段：`comfyui_url` 填本地 ComfyUI 服务地址，默认值 `http://127.0.0.1:8188`；填好后点击「测试连接」确认服务可达。`comfyui_api_key` 为可选字段，来自 Comfy Platform（platform.comfy.org/profile/api-keys），不是使用本地 ComfyUI 的必需项。Docker 部署取宿主 ComfyUI 时，地址应写 `host.docker.internal:8188`（macOS 与 Windows）或宿主机 IP（Linux），该写法为镜像内网络通信所需。ComfyUI 本身的安装步骤放在第 9 章，此处只需先把地址配置好。

RunningHub 侧配置三个字段：`runninghub_api_key` 为在 RunningHub 平台获取的密钥，使用 `runninghub/` 前缀工作流时必填；`runninghub_concurrent_limit` 为并发限制，取值 1 到 10，普通会员默认为 1，调大可并行处理多个分镜但需确认会员级别允许（v0.1.14 前后版本起该值可配置）；`runninghub_instance_type` 为实例类型，留空表示 24GB 显存机型，设置为 `plus` 表示使用 48GB 显存机型，官方建议大尺寸视频生成使用 48GB 机型，该机型费用更高。

3.4 直连 API 媒体模型配置

「API 媒体模型配置」区的用途是不经 ComfyUI 与 RunningHub，直接调用模型供应商的图像、视频与素材分析能力。该配置区对应 config.yaml 中 `api_providers` 段。需要注意版本时点：该区为 2026 年 6 月主分支更新引入，截至核实日未包含在 v0.1.15 整合包内，使用它需要拉取主分支最新代码运行，或采用包含该特性的更晚发布包。

官方支持的供应商与鉴权字段如下，Base URL 在 Web 界面中提供官方默认值，一般不需修改。

供应商	用途	鉴权字段	默认 Base URL（config.yaml 申明）
OpenAI	GPT 图像生成	`api_key`	`https://api.openai.com/v1`
DashScope（通义、WAN、HappyHorse）	通义万象图像与视频	`api_key`	`https://dashscope.aliyuncs.com/api/v1`
ARK / Seedream / Seedance	字节图像（Seedream）与视频（Seedance）	`api_key`	`https://ark.cn-beijing.volces.com/api/v3`
Kling（可灵）	视频生成	`access_key`、`secret_key`	`https://api-beijing.klingai.com`

公共配置区（`common`）包含两项与调试及网络有关的字段，对所有供应商生效：

- `local_proxy`：本地代理地址，例如 `http://127.0.0.1:9090`；每个供应商另有 `use_proxy` 字段可单独决定是否启用代理；
- `print_model_input`：调试开关，开启后终端会打印实际发送给模型的提示词、模型名称与输入文件路径，用于排查画面质量与请求参数问题。

只使用 ComfyUI 或 RunningHub 的使用者无需填写本区；仅当选择了 `api/` 前缀的工作流时，对应供应商的密钥才是必填项。

3.5 配置文件结构总表

config.yaml 的四个顶层段与对应章节如下，供查阅与手工编辑时定位。

段名	内容	对应章节
`llm`	文案模型的密钥、地址、模型名	3.1
`api_providers`	直连 API 供应商与代理、调试开关	3.4
`comfyui`	ComfyUI 地址、RunningHub 密钥与机型、图像、视频、TTS 默认工作流与提示词前缀	3.3、第 7 章、第 9 章
`template`	默认视频模板路径，如 `"1080x1920/image_default.html"`	第 8 章

配置完毕后，点击 Web 界面中的「保存配置」。验证方式是回到生成页面，确认各下拉框已能列出工作流与模板，随后进入第 4 章。

第 4 章 生成第一个视频：三栏 Web 界面完整走读

本章按一次完整生成的顺序走遍全部界面控件。目标读者在完成本章后应能独立设定主题、语音与画面方案，并拿到一段可播放的成片。界面为三栏布局：左侧栏是内容输入，中间栏是语音与视觉设置，右侧栏是生成与预览；系统配置与内置 FAQ 位于侧边栏。以下操作以通义千问加 RunningHub 默认工作流为示例配置，其他组合同理。

4.1 步骤一：确认配置与提示条件

展开侧边栏「系统配置」面板，确认三项：LLM 配置已填入密钥并通过连接测试；画面生成配置（ComfyUI 地址或 RunningHub Key）已填写；点击「保存配置」成功。若尚未完成，回到第 3 章。

两处环境提示需要提前说明。其一，模板画面渲染依赖本机 Chrome 内核，若终端出现找不到 Chrome 浏览器程序的报错，安装 Google Chrome 即可恢复，该问题与解决方案来自官方 FAQ。其二，若使用本地（selfhost）工作流，官方在 v0.1.15 起会弹出提示，建议先在 ComfyUI 中验证该工作流能正常运行后再接入生成，避免生成中途报 400 错误（详见第 9 章）。

4.2 步骤二：内容输入

左栏「内容输入」区分生成模式与 BGM 两块。

生成模式二选一：

- 「AI 生成内容」：在文本框输入主题，由 AI 自动创作完整文案。首次体验建议输入一个信息密度明确的话题，官方 README 与官方文档中的示例为「为什么要养成阅读习惯」，其他可用示例包括「如何提高工作效率」「健康饮食的重要性」「旅行的意义」；
- 「固定文案内容」：粘贴现成稿件，跳过 AI 写作。该模式可选择分割方式，详见第 5 章。

「场景数量」（分镜数量）可设定，默认为 5 个分镜；数量越多，生成时间与 API 消耗越大。

BGM 设置三选一：「无 BGM」保留纯人声；「内置音乐」选择预置曲目（如 default.mp3）；「自定义音乐」需先将 MP3 或 WAV 文件放入项目 `bgm/` 目录（Docker 部署放入 `data/bgm`，见 2.3 节），该文件随后会出现在下拉列表中。点击「试听 BGM」可在生成前确认音色与节奏。

4.3 步骤三：语音与视觉设置

中间栏分「语音设置」与「视觉设置」两块。

语音设置包含 TTS 工作流选择与可选的参考音频上传。TTS 工作流下拉列表自动扫描 `workflows/` 目录中的语音工作流，默认的 Edge-TTS 方案即可满足首次体验；参考音频用于声音克隆，属于进阶用法，详见第 6 章。此栏还提供「预览语音」功能：输入一小段测试文本，点击预览即可在不启动完整生成的情况下试听合成效果。

视觉设置包含三控件：

- **图像生成工作流**：决定配图的模型与风格。默认为 `runninghub/image_flux.json`；图像尺寸默认 1024x1024，可整体调整，注意不同模型对尺寸有各自的限制；提示词前缀控制画面整体风格，输入英文，官方默认值为 `Minimalist black-and-white matchstick figure style illustration, clean lines, simple sketch style`（极简黑白火柴人简笔风格）；点击「预览风格」可用当前配置快速试产出图；
- **视频模板**：决定画面版式。下拉按尺寸分组显示（竖屏 1080x1920、横屏 1920x1080、方形 1080x1080），首次体验选择竖屏分组的默认模板即可。模板类型与命名规则详见第 8 章；
- **API 视频生成**（选用动态画面时）：当选择 `video_` 模板或包含视频工作流时，可在此选择直连视频模型，相关配置见第 10 章。

4.4 步骤四：执行生成

点击右栏「生成视频」按钮。右栏会显示实时进度，官方给出的流程顺序为：生成文案、逐分镜生成配图、合成语音、合成视频；进度文本会精确到分镜，例如「分镜 3/5 - 生成插图」。

生成耗时的官方估计：5 个分镜的视频为 2 至 5 分钟，具体取决于 LLM 响应速度、图像生成途径与网络状况。使用 RunningHub 且将并发限制设为大于 1 的数值，多个分镜可并行出图，等待时长相应缩短。

4.5 步骤五：预览与取用成片

生成完成后，右栏自动出现视频播放器，并显示视频时长、文件大小、分镜数量与视频尺寸。所有成片保存在项目根目录的 `output/` 文件夹，文件可直接取用或转推至发布平台。

首次生成结果不理想属于正常现象，调优思路在第 5 至第 8 章分别展开：换一个 LLM 看文案结构、换提示词前缀与图像尺寸看画面风格、换 TTS 工作流与参考音频看声音、换模板看版式。四类因素彼此独立，试错成本很低。

第二部 进阶篇：打磨内容与视听风格

第 5 章 内容创作：AI 文案、固定文案与自定义素材

文案决定一条视频的信息质量，也是后续所有画面与语音的源头。生成模式选错会带来方向性返工：想自己把控叙事却误用 AI 写稿，或想快速成片却迫不得已逐字写稿。本章说明两种生成模式的适用差异、文案分割规则，以及 2025 年 12 月起加入的自定义素材能力。

5.1 两种生成模式的选择

两种模式的差异与适用判断如下。

维度	AI 生成内容	固定文案内容
输入	一句主题词或短句	完整解说稿件
分镜划分	LLM 结构化输出决定	按所选分割方式切分
返改成本	换主题或换模型重跑	改稿后重跑
适用场景	快速产出、无现成稿	已有稿件、精确控制表达

使用 AI 模式时的主题写法直接影响文案质量。官方文档与社区示例均为包含明确主体与设问角度的短句，例如「为什么要养成阅读习惯」「如何理解反脆弱」。主题的信息密度越高，LLM 的结构化输出越稳定。

5.2 AI 模式：分镜数量与文案结构

分镜数量在界面上默认为 5，REST API 参数 `n_scenes` 的合法范围为 1 到 20，仅对 AI 生成模式生效。数量与时长、消耗大体线性相关：分镜越多，文案越长，出图与语音调用次数越多。

该项目在 2025 年 12 月的更新中优化了 LLM 返回结构化数据的逻辑，即要求模型按固定格式（标题、分镜编号、每镜解说文本、画面描述）输出，这是界面能直接渲染分镜面板的原因。若某模型输出不稳定，处理办法是更换更稳的模型或重试，属于该模式固有的概率性行为。

5.3 固定文案模式：分割方式

固定文案模式提供三种分割方式（2025 年 12 月版本加入）：

分割方式	规则	典型用途
按段落分割	以空行为分隔符，每段一个分镜	每镜承载完整语义段的长稿
按行分割	以换行符为分隔符，每行一个分镜	已按节奏分行的台本
按句子分割	智能识别句子边界，每句一个分镜	未排版的一整段文本

选择时应对照稿件形态：稿件本身有序则选段落或行，让既有结构映射为分镜；无结构的整段文字选句子分割，交给系统切分。

5.4 自定义素材：用现有照片与视频生成脚本

2025 年 12 月版本加入「自定义素材」功能：使用者上传自己的照片与视频，AI 先分析素材内容，再生成与之匹配的解说脚本并继续走后续流水线。该功能改变了本工具「先文案后画面」的默认方向，适合手中已有一批实拍或截图素材、只补解说与成片的场景。素材上传入口位于 Web 界面左侧内容输入区内；在直连 API 视频方案下，该工作流生成的 API 视频片段会按旁白音频时长对齐，并参考相邻片段的信息以提升连贯性，这一点由 2026 年 6 月的更新说明确认。

5.5 文案调优建议

以下为建议类内容，理由附后。

- **重跑与多次生成**：LLM 输出存在随机性，同一主题多次生成再择优，成本低于逐句人工改稿；
- **更换文案模型**：不同模型的行文风格与结构稳定性差异明显，官方将通义千问列为性价比推荐方案；换模型的操作成本仅在一处配置（第 3 章）；
- **修正主题表述**：把含糊主题改写为「主体、角度、期望立场」具备的短句，比盲目增加分镜数量更能改善文案。

第 6 章 语音体系：TTS 工作流、试听预览与声音克隆

解说声音的质感对完播率影响显著，且语音层是四层流水线中最容易单独试错的一层：语音设置区提供测试文本与预览按钮，调整无需重跑全流程。本章覆盖 TTS 工作流的挑选、预览用法、参考音频的声音克隆，以及背景音乐。

6.1 TTS 工作流的结构与默认方案

界面上的 TTS 下拉列表自动扫描 `workflows/` 目录：`selfhost/` 下的条目走本地 ComfyUI，`runninghub/` 下的条目走云端。默认工作流为 `selfhost/tts_edge.json`，即 Edge-TTS：直接使用微软公开的免费语音接口，无需本地模型，但接口受网络波动影响，失败频率可能上升。

官方 FAQ 针对这一问题的建议是：网络环境明显不稳时，切换为基于 ComfyUI 的 TTS 工作流（即 `tts_` 前缀且安装了相应语音节点的方案），稳定性更高，理由是这类工作流在本机或 RunningHub 设备上完整运行，绕开了外部公开接口。2026 年 1 月版本另补充了多语言 TTS 音色支持，不同音色所支持的语言以工作流内模型为准。

6.2 预览语音的用法

语音设置的预览功能按以下顺序使用：在测试文本框输入与最终解说风格相近的短句（断句、标点、生僻词应与正稿一致，预览效果才有代表性）；点击「预览语音」试听；满意后保持当前工作流，不满意则切换工作流或上传参考音频后重新预览。

6.3 声音克隆：参考音频

部分 TTS 工作流支持声音克隆：以上传的参考音频作音色样本，合成出与样本相近的人声，代表性方案为 Index-TTS。Edge-TTS 不支持克隆，需要一个支持克隆的工作流（如 Index-TTS）；官方文档对参考音频的质量要求为：MP3、WAV、FLAC 均可，时长建议 10 至 30 秒，内容避免背景噪音。克隆链路为：上传参考音频，界面试听确认，预览语音验证效果，最后正常生成视频。参考音频质量直接决定克隆效果。

6.4 背景音乐与音量

背景音乐来源有三种：不使用、内置曲目（`bgm/` 目录预置，如 default.mp3）、自定义曲目（放入 `bgm/` 后在下拉中出现；Docker 部署放 `data/bgm`）。界面上「试听 BGM」用于生成前确认。

音量层面，REST API 参数 `bgm_volume` 控制 BGM 相对人声的响度，取值 0.0 到 1.0，默认 0.3。默认值下 BGM 作为衬底不干扰解说；若解说录音动态范围较小，可小幅下调 BGM 音量保持人声清晰，该值属于建议试听后微调的参数。

第 7 章 视觉体系：图像工作流、尺寸与提示词前缀

画面层由三个相互独立的控件共同决定：图像生成工作流、图像尺寸、提示词前缀，再叠加第 8 章的模板。三者各自可单独试错，配合界面上的「预览风格」功能，可以在不跑完整流程的情况下检验画面方案。本章给出每个控件的变化范围与组合建议。

7.1 图像生成工作流的选择

图像工作流下拉列表包含三类条目，前缀决定其运行途径：

前缀	途径	前提	代表条目
`selfhost/`	本地 ComfyUI	本机 ComfyUI 服务在线	`selfhost/image_flux.json`
`runninghub/`	RunningHub 云端设备	RunningHub API Key	`runninghub/image_flux.json`（官方推荐默认）
`api/`	直连模型供应商	对应供应商密钥（第 3.4 节）	依供应商而变的图像工作流

默认工作流在 config.yaml 的 `comfyui.image.default_workflow` 字段设置，示例文件给出注释：无本地环境时推荐 `runninghub/image_flux.json`（即 FLUX 系列模型）。图像模型除 FLUX 外还包括 Stable Diffusion 系列，具体可列举的模型由 `workflows/` 目录中的实际文件决定。

7.2 图像尺寸

图像尺寸以像素为单位在界面调节，默认 1024x1024。尺寸选择应与目标模板或视频尺寸匹配，官方文档给出如下对照。

尺寸	用途
1024x1024	方形，适合小红书等图文平台
1080x1920	竖屏，适合抖音、快手类短视频
1920x1080	横屏，适合 B 站、YouTube

尺寸上限由模型能力决定，不同模型对可生成的最长边与总像素有各自限制，超出会直接报错或被服务拒绝；界面或日志报错时优先核对目标模型的尺寸要求后回落调整。RunningHub 运行大尺寸视频生成时，官方提示使用 48GB 显存机型（`runninghub_instance_type: "plus"`）。

7.3 提示词前缀：风格总开关

提示词前缀（Prompt Prefix）是一段拼接在每个画面描述之前的英文风格词，控制整条视频的视觉一致性。工作方式为：LLM 产出的每个分镜画面描述，都会拼上此前缀再送入图像模型。修改入口有两处，界面中的提示词前缀输入框，或 config.yaml 中 `comfyui.image.prompt_prefix` 字段。

默认值与效果：`Minimalist black-and-white matchstick figure style illustration, clean lines, simple sketch style`，产图为极简黑白火柴人简笔画风，见于官方默认配置。操作步骤为：输入新前缀，点击「预览风格」检查单帧效果，确认后生成。前缀要求使用英文，这一点由官方使用说明明确，中文输入会削弱风格词的引导效果。

自定义前缀的写法属于提示词工程范畴，官方教程以两层结构给出建议：题材域加风格特征。例如「Minimalist black-and-white illustration, clean lines, simple style」即为题材域（极简黑白插画）加风格特征（线条干净、风格简洁），官方教程示例。配套变量是正稿的画面描述质量，若描述本身含糊，前缀再精确也难以拉齐风格。

另有一项与画面描述相关的辅助能力：项目对图片与视频的反推工作流（由已有图像生成提示词，便于以参考图反向定制风格词）做过优化（2025 年 12 月更新），具体参数以工作流文件与后续官方文档为准。

7.4 与模板的联动

选择 `image_` 前缀模板时，出图作为模板背景填充版式；因此方案确认顺序应为先定模板版式，再调图像工作流与前缀，最后用「预览风格」核对整帧效果。若选用 `video_` 模板，画面背景来自第 10 章的动态视频生成，静态图的设置不再影响成片背景。

第 8 章 视频模板：模板体系与画面版式控制

模板决定每个分镜的文字排版、背景处理方式与整体观感，同一份文案与配图套用不同模板，成品差异直观可见。模板按尺寸目录与用途前缀组织，理解命名规则后，挑选与自制均不需要编程经验；有 HTML 基础的使用者可以完全自制。本章覆盖模板体系、可用模板、预览方法与自制模板。

8.1 模板命名与目录结构

模板是位于 `templates/` 目录的 HTML 文件，按视频尺寸存放于三个子目录；文件名前缀与其对应的媒体依赖如下。

前缀	类型	媒体依赖	成本特征
`static_`	静态模板	无需 AI 生成媒体，纯文字样式渲染	最快，无媒体生成消耗
`image_`	图像模板	需 AI 生成背景图，走图像生成工作流	每分镜一次出图调用
`video_`	视频模板	需 AI 生成背景视频，走视频生成工作流	耗时与消耗最高

目录结构如下。

```
templates/
├── 1080x1920/   # 竖屏
│   ├── static_*.html
│   ├── image_*.html
│   └── video_*.html
├── 1920x1080/   # 横屏
│   └── image_*.html
└── 1080x1080/   # 方形
    └── image_*.html
```

8.2 主要内置模板

官方模板页面列出的模板数量较多，此处摘出代表性的条目。完整列表与实拍效果图见官方文档站的模板页（aidc-ai.github.io/Pixelle-Video/zh/user-guide/templates/）。

模板	尺寸	官方描述的适用内容
`static_default`、`static_excerpt`	竖屏	默认静态版式；图文摘抄
`image_default`	竖屏	简洁通用默认模板
`image_simple_line_drawing`	竖屏	简笔画，适合认知成长类
`image_health_preservation`	竖屏	养生科普
`image_simple_black`	竖屏	极简黑色背景，适合心灵鸡汤类
`image_full`	竖屏与横屏均有	全屏版式，适合书单号
`image_healing`、`video_healing`	竖屏	治愈、疗愈类内容，动态版为视频模板
`image_film`	横屏	电影风格，沉浸式体验
`image_wide_darktech`	横屏	暗黑科技风，适合技术与游戏内容
`image_minimal_framed`	方形	极简边框，适合社交媒体分享

界面上模板下拉按尺寸分组，并有模板预览画廊协助选择；选定后可用「预览模板」填入自定义参数试版式。

8.3 模板自定义参数

模板支持从外部传入自定义参数（config.yaml 与 REST API 中的 `template_params` 字段），官方 REST API 文档示例给出两个参数样例：`accent_color`（主题色，如 `#3498db`）与 `background`（自定义背景地址）。模板具体接受哪些参数取决于模板自身的实现，调整后在预览中验证即可。

8.4 自制模板

自制不需要接触项目代码，步骤为：从 `templates/` 目录复制一个现有模板文件到目标尺寸目录；修改其中的 HTML 与 CSS；以合法名称保存（保留类型前缀）；界面下拉中即会出现新模板。模板在渲染时可读取以下变量（Jinja2，一种 Python 模板引擎语法）：

变量	含义
`{{ title }}`	视频标题（可选）
`{{ text }}`	当前分镜的解说文本
`{{ image }}`	当前分镜的配图（如有）

官方示例模板如下，为最简竖屏静态版式，可直接以此为基础改制。

```
<!DOCTYPE html>
<html>
<head>
    <style>
        body {
            width: 1080px;
            height: 1920px;
            margin: 0;
            background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
            display: flex;
            align-items: center;
            justify-content: center;
            font-family: 'Arial', sans-serif;
        }
        .content {
            text-align: center;
            color: white;
            padding: 40px;
        }
        .text {
            font-size: 48px;
            line-height: 1.6;
        }
    </style>
</head>
<body>
    <div class="content">
        <div class="text">{{ text }}</div>
    </div>
</body>
</html>
```

官方给出的制作规范有四条，均为硬性或强建议：

1. **画布尺寸一致**：模板 body 的宽高必须与目标视频尺寸完全一致（竖屏 1080x1920、横屏 1920x1080、方形 1080x1080），否则渲染出的画面尺寸不正确；
2. **文本排版**：字号与行高需保证可读性，为文字添加阴影或背景块提高对比度，并控制文本长度避免溢出；
3. **图片处理**：背景图使用 `object-fit: cover` 保证满幅填充，必要时叠加渐变或遮罩层保证文字可读，考虑图片加载失败的降级方案；
4. **性能**：避免复杂 CSS 动画，压缩背景图片体积，使用系统字体或 Web 安全字体。

模板渲染依赖本机 Chrome 内核（见 4.1 节的说明），自制样式复杂脚本使用过多时，同样会遇到该环境的渲染差异问题。

第三部 高级篇：定制、扩展与自动化

第 9 章 ComfyUI 本地部署与工作流定制

第二部中的四类设置都不必接触工作流内部；本章打开这一层。设计目标有二：其一，让使用者在没有本地 GPU 或不便维护本地 ComfyUI 时，仍能主导生成能力的选择；其二，讲解 ComfyUI 官方运行方式与本项目工作流的组织规则，为替换默认模型或编写自定义工作流做准备。

9.1 ComfyUI 的角色与安装

ComfyUI 是基于节点连线方式的生成式 AI 图形编排环境。图像、视频、语音三类生成任务都以「工作流」（Workflow，即描述模型、节点与参数的 JSON 文件）形式在其中运行；Pixelle-Video 对这些工作流做了基于命名与参数绑定的集成，使得界面操作可以直接驱动 ComfyUI 中的生成。

ComfyUI 的安装方式（官方文档说明）：

```
$ git clone https://github.com/comfyanonymous/ComfyUI.git
$ cd ComfyUI
$ pip install -r requirements.txt
```

启动：

```
$ python main.py
```

服务默认监听 `http://127.0.0.1:8188`。模型文件需依照 ComfyUI 官方说明下载并放到其模型目录；本地生成图像建议 NVIDIA 显卡显存达 6GB 以上。启动后回到 Pixelle-Video，在系统配置中确认 ComfyUI 地址与「测试连接」通过。ComfyUI 检查与常用插件问题，优先以 ComfyUI 官方支持渠道为准。

9.2 工作流的目录与命名规则

本项目内置的工作流位于 `workflows/` 目录，分为两组、三类前缀：`selfhost/`（本地 ComfyUI 用）与 `runninghub/`（RunningHub 云端用）；前缀 `image_`、`video_`、`tts_` 分别对应图像、视频与语音三类任务。内置代表条目：

文件	用途	运行途径
`selfhost/tts_edge.json`	语音合成（默认）	本地
`runninghub/image_flux.json`	图像生成（官方推荐默认）	云端
`selfhost/image_flux.json`	图像生成	本地
`runninghub/video_wan2.1_fusionx.json`	视频生成（官方推荐默认，基于 WAN 2.1 模型）	云端
`selfhost/video_wan2.1_fusionx.json`	视频生成	本地

官方对视频工作流还有以下说明：视频生成为新加入的能力，本地工作流需要在本机安装相应的视频生成节点；云端条目无需本地环境。界面在启动本地（selfhost）工作流时会弹出提醒，建议先在 ComfyUI 中确认该工作流能正常出图后再接入生成流程，以避免生成中途出现 400 错误（该提醒自 v0.1.15 起）。

9.3 自定义工作流的接入规范

自定义工作流（官方 FAQ 给出的规范）按以下四步接入：

1. **本地跑通**：先在本地 ComfyUI 中确认工作流可正常生成；
2. **参数绑定**：找到需要由程序动态写入提示词的文本输入节点（如 CLIP Text Encode），将该节点的标题（Title）改为 `$prompt.text!` 或 `$prompt.value!`（按节点接受的输入类型选择其一）。程序运行时会按此标题识别并注入提示词。格式参考 `workflows/selfhost/` 现有文件；
3. **导出格式**：在 ComfyUI 中使用 Save (API Format) 把工作流导出为 API 格式 JSON；
4. **落盘命名**：把导出文件放入 `workflows/` 目录，并按用途加前缀，例如图像用 `image_my_style.json`，视频用 `video_`，语音用 `tts_`。保存后 Web 界面下拉中即会出现该工作流。

将内置 RunningHub 工作流拉回本地调试的做法（官方 FAQ）：打开对应 JSON 找到其中的工作流 id，把 id 拼接为 RunningHub 网址打开该工作流页面，下载 JSON 并拖入本地 ComfyUI 画布调试。该方法用于检查云端工作流在本地设备上的兼容性。

9.4 部署形态的取舍

形态	优点	代价
本地 ComfyUI	API 调用免费；数据不出本机	需维护环境与下载模型；受本机显存限制
RunningHub 云端	无本地环境，随用随开	按积分消耗费用；并发受会员等级限制
直连 API（第 10 章）	无需工作流调试	全部按第三方平台价格计费

第 10 章 直连 API 媒体模型与 API 视频生成

不开 ComfyUI、不注册 RunningHub，直接使用模型供应商的官方接口，是三条画面生成途径中最简单的一条。本章说明哪些供应商可以接入、配置方法、参数面板的能力差异，以及动态视频生成中与审核、重试相关的行为。

10.1 能力范围与版本说明

直接调用图像与视频模型的能力由 2026 年 6 月的主分支更新引入，内容为在 WebUI 中配置图像与视频模型供应商、Base URL 与代理开关；截至核实日未随 v0.1.15 整合包发布。使用本部分功能需要满足其一：拉取主分支最新代码自行运行，或使用包含该特性的更晚版本。官方 README 中标注的供应商能力如下。

供应商	图像能力	视频能力
OpenAI	GPT 图像（GPT Image）	—
DashScope（阿里）	通义万象图像	通义万象视频（WAN、HappyHorse）
ARK（火山引擎）	Seedream 图像	Seedance 视频
Kling（可灵）	—	可灵视频

10.2 配置方法

配置入口为 Web 界面「系统配置」下的「API 媒体模型配置」区，对应 config.yaml 的 `api_providers` 段。每个供应商的三个配置项：鉴权信息（`api_key`；Kling 为 `access_key` 与 `secret_key`）、`base_url`（界面提供官方默认地址）、`use_proxy`（是否启用本地代理，代理地址写在公共 `local_proxy` 字段，详见第 3.4 节）。需要排查画面生成质量时，开启公共区的「打印模型请求参数」开关（`print_model_input`），终端会输出实际发送的提示词、模型名与输入文件路径。

10.3 API 工作流与视频生成参数

使用时在图像或视频工作流下拉中选择 `api/` 前缀条目。官方更新说明确认，选用支持动态视频的模板或扩展工作流时，界面会按所选模型的能力显示对应参数项，包括分辨率、画幅比例、时长、水印与原生音频等。这意味着不同模型的具体参数面板会有差异，以所选模型实际显示的选项为准。

可靠性层面，官方 README 记载了两项内置机制：API 返回的网络下载支持重试；若触发平台内容审核失败，系统会自动以中性化后的提示词重试一次。审核与重试的平台规则最终以各家供应商政策为准。

10.4 与自定义素材的协同

在「自定义素材」工作流（第 5.4 节）中使用直连 API 视频时，官方 README 说明系统会尽量按旁白音频时长生成对应片段，并参考相邻片段的信息保持画面连贯性。该行为使素材型视频在分段生成时保持续接感，无需逐段手工衔接。

10.5 三条途径的选择建议

以下为建议类内容，理由附后。

- **图像生成选 RunningHub 或本地 ComfyUI**：出图消耗大且使用频繁，积分制平台长期费用通常高于本地一次投入；无本地显卡且不愿注册过多平台时，选直连 API 的图像供应商最简单；
- **动态视频按场景分工**：偶发使用 video 模板或做单条演示，用直连 API 最省事；批量、反复的生产线，配置好 RunningHub 云端工作流的并发与机型更划算；
- **同一项目可混合使用**：图像走 RunningHub、语音走本地、视频走直连 API 完全合法，逐项替换即可。

第 11 章 扩展流水线：数字人口播、图生视频、动作迁移

前三章都围绕「解说配画面」主干流水线；本章覆盖四条扩展流水线。它们的共同点是改变了输入或输出的形态：输入可以是真人形象、既有图像或自有素材，输出可以是数字人播报或由静态图延展出的动态片段。以下能力描述与版本时点均来自官方 README 与发布说明。

11.1 数字人口播

数字人指由 AI 生成的讲解人物形象，由其承担画面中的播报角色的流水线称数字人口播。该项目于 2026 年 1 月 14 日的版本更新中加入该流水线，官方展示样例包括韩语数字人口播，说明该模块可输出带多语言的形象播报视频。使用前提：数字人模块依赖视频生成模型（如 WAN 2.1 相关工作流）或直连 API 视频服务，本地显存不足时优先使用 RunningHub 48GB 机型或直连 API。

11.2 图生视频

图生视频指输入一条静态图像，由模型延展出短动态片段的能力。2026 年 1 月 14 日版本把该形制作为独立流水线加入，官方样例为卡通视频。它与「以图作背景」的根本差异在于：背景图在身体动作与镜头运动上完全静止，而图生视频的片段带真实运动，适合有镜头感的场景。本质上是把宽泛的「静态画面」换成「连续的运动画面」，其余流水线不变。

11.3 动作迁移

动作迁移解决「让一个特定人物做出指定动作」的问题：输入一段参考视频与一张参考图（如一张人物或角色图），系统把参考视频中的动作迁移到图中角色。2026 年 1 月 26 日版本加入该功能，官方样例为跳舞小猫。设计者由此可以在不训练模型的情况下生成特定角色动作的动画片段。

11.4 自定义素材流水线

该能力已在 5.4 节介绍：上传自有照片与视频，AI 分析素材并生成脚本，再把素材与 AI 生成片段结合合成成片。在直连 API 视频方案下，片段长度会按旁白音频时长对齐。它与其他三条的差异在于生成方向：前三条扩展的是画面生成能力，这一条扩展的是文案与素材的输入能力。

11.5 扩展模块的通用提示

扩展模块在 Web 界面中以独立功能面板提供入口；具体操作与可选参数以界面面板提示与官方文档为准，本章只给出能力定义与前提条件。以下为风险与建议类内容，理由附后。

- **显存与机器选择**：数字人、图生视频、动作迁移均为视频级生成，显存消耗明显高于静态出图；本地配置不足时改用 RunningHub 48GB 机型（`runninghub_instance_type: "plus"`）或直连 API。若工作流本地运行卡死或失败，先核对显存占用；
- **素材合规**：上传他人肖像做数字人或动作迁移前，应确认已取得授权，并遵从平台发布规范；生成式内容用于商业发布时，还需符合发布平台的标识要求。

第 12 章 精通实战：批量生成、历史记录、性能优化与故障排查

前十一章覆盖界面操作与定制；本章面向把 Pixelle-Video 用成日常生产工具的需求：用程序接口自动批量生产、管理历史成片、系统化性能调优与故障定位。完成本章后，读者应当能编写少量 Python 调用代码即可完成内容批次生产，并具备把故障定位到环节的基本方法。

12.1 Python SDK：以代码驱动生成

项目提供 Python SDK。SDK 的核心服务类为 PixelleVideoCore，位于 `pixelle_video.service` 模块；使用方式为初始化后调用 `generate_video` 方法。官方快速开始示例照录如下：

```python
from pixelle_video.service import PixelleVideoCore
import asyncio

async def main():
    pixelle = PixelleVideoCore()
    await pixelle.initialize()

    result = await pixelle.generate_video(
        text="为什么要养成阅读习惯",
        mode="generate",
        n_scenes=5
    )
    print(f"视频已生成: {result.video_path}")

asyncio.run(main())
```

`generate_video` 的主要参数如下（来源为官方 API 概览文档）：

参数	类型	说明
`text`	str	主题或完整文案
`mode`	str	`"generate"` 为 AI 生成，`"fixed"` 为固定文案
`n_scenes`	int	分镜数量，仅 generate 模式有效
`title`	str, 可选	视频标题；不填则由系统生成
`tts_workflow`	str	TTS 工作流
`media_workflow`	str	媒体生成工作流（图像或视频）
`frame_template`	str	视频模板路径
`template_params`	dict, 可选	模板自定义参数
`bgm_path`	str, 可选	BGM 文件路径
`bgm_volume`	float	BGM 音量，0.0 至 1.0

返回值为 `VideoResult` 对象，其 `video_path` 字段为成片路径。方法为异步接口，需配合 asyncio（Python 标准库中的异步编程模块）运行。

12.2 REST API：跨语言与跨机调用

REST API 是 HTTP 接口层的同名能力，官方示例启动命令为：

```
$ uv run uvicorn api.app:app --host 0.0.0.0 --port 8000
```

uvicorn 是 Python 中常用的 ASGI 服务器，负责把 FastAPI 应用发布为 HTTP 服务；Docker 部署中该服务即 8000 端口的 `pixelle-video-api` 容器（见 2.3 节）。接口层提供同步与异步两种生成方式，Swagger UI（接口文档浏览器页面）位于 `http://localhost:8000/docs`，接口字段与响应同样以该页面为准。

同步生成 `POST /api/video/generate/sync`：等待完成后返回结果，适合 30 秒以内的小视频。请求体示例：

```
{
  "text": "为什么要养成阅读习惯",
  "mode": "generate",
  "n_scenes": 5,
  "frame_template": "1080x1920/image_default.html",
  "template_params": {
    "accent_color": "#3498db",
    "background": "https://example.com/custom-bg.jpg"
  },
  "title": "阅读的力量"
}
```

成功时的响应包含视频地址、时长与文件大小，示例：

```
{
  "success": true,
  "message": "Success",
  "video_url": "http://localhost:8000/api/files/xxx/final.mp4",
  "duration": 45.5,
  "file_size": 12345678
}
```

异步生成 `POST /api/video/generate/async`：立即返回任务标识，适合长视频或批量提交。响应形如 `{"success": true, "task_id": "abc123"}`；随后以 `GET /api/tasks/{task_id}` 查询，任务完成后响应中的 `result` 字段指向成片地址。

请求参数总表（含官方给出的可选字段）：

参数	类型	必填	说明
`text`	string	是	主题或完整文案
`mode`	string	否	`"generate"` 或 `"fixed"`
`n_scenes`	int	否	分镜数量（1-20），仅 generate 模式有效
`title`	string	否	标题
`frame_template`	string	否	模板路径，如 `1080x1920/image_default.html`
`template_params`	object	否	模板自定义参数
`media_workflow`	string	否	媒体工作流
`tts_workflow`	string	否	TTS 工作流
`ref_audio`	string	否	声音克隆参考音频路径
`prompt_prefix`	string	否	图像风格前缀
`bgm_path`	string	否	BGM 文件路径
`bgm_volume`	float	否	BGM 音量，默认 0.3

12.3 批量生产与历史记录

批量生产的两条路径。其一，Python 或其他语言的调用方循环向异步接口提交任务，按返回的 task_id 轮询完成状态；文本差异与模板差异都可以放入循环变量。其二，Web 界面自带批量创建视频任务的功能（2025 年 11 月版本加入），适合不写代码的批量需求。

历史记录页面同期加入：可查看全部生成过的视频、每条记录的生成参数与详细信息，便于回溯某成片完整参数或复跑相似主题。成片文件本身位于 `output/` 目录，磁盘紧张时按日期迁移。

12.4 性能优化

官方故障排查文档给出的生成速度优化手段包括：使用本地 ComfyUI、减少分镜数量、更换更快的 LLM、检查网络。以下按环节归纳，并补充替代方案的建议及其理由。

环节	优化方向	理由
文案生成	选响应快的 LLM 预设	文案是流水线第一步，直接决定后续等待
图像生成	本地 ComfyUI 或调大 RunningHub 并发	分镜出图可并行；官方文档将本地 ComfyUI 列为推荐的速度优化手段
语音生成	切换稳定 TTS 工作流	Edge-TTS 受网络波动影响，不稳定时切换降低失败重试的耗时
视频合成	确认 Chrome 已安装、模板不叠重动画	合成阶段调用渲染，模板性能影响见第 8.4 节

12.5 故障排查

从官方故障排查文档与 FAQ 整理的排查路径如下，按环节定位。

- **依赖安装失败**：先清理缓存再重新安装，官方给出命令 `uv cache clean` 后接 `uv sync`；
- **ComfyUI 连接失败**：确认 ComfyUI 已运行；核对地址是否为 `http://127.0.0.1:8188`；在浏览器直接访问该地址验证；排查防火墙；
- **LLM 调用失败**：常见原因依次为：Base URL 多出后缀或空格、Key 无效或余额不足、模型名称拼写有误；核对以所用供应商的官方 API 文档为准；
- **TTS 失败**：Edge-TTS 在网络波动下失败概率上升，或切用 ComfyUI TTS 工作流；声音克隆失败时核对参考音频格式与时长（第 6.3 节）；
- **找不到 Chrome 内核**：安装 Google Chrome（见 4.1 节）；
- **视频生成失败**：核对工作流文件存在、模型已下载、磁盘与内存充足；本地工作流先在 ComfyUI 内单独跑通再接入；
- **日志位置**：项目根目录的 `api_server.log`（API 服务日志）与 `test_output.log`（测试日志）可提供定位信息，来源为官方故障排查文档。

附录 A 常见错误速查表

下表汇总正文各章出现的错误信息与处理方法，按出现环节排序。

错误或现象	原因	处理
浏览器未打开 8501 页面	服务未启动或端口被占用	确认启动命令执行成功；核对端口进程；Windows 整合包双击 start.bat，源码安装执行 streamlit 启动命令
找不到 Chrome 浏览器程序	本机缺少 Chrome 内核	安装 Google Chrome
LLM 连接失败、输出报错	Key 无效、地址多余后缀或空格、余额不足	用界面的连接测试按钮核对；对照供应商 API 文档
TTS 生成失败或频率性失败	Edge-TTS 免费接口受网络波动	切换 `tts_` 前缀的 ComfyUI TTS 工作流
出图报 400 错误	本地（selfhost）工作流未在 ComfyUI 中跑通	先在 ComfyUI 内验证工作流（v0.1.15 起有弹窗提醒）
图像尺寸报错	超出模型支持的尺寸上限	核对目标模型尺寸要求并回落调整
模板文本溢出或错位	文本过长或字号行高不当	按第 8.4 节排版规范调整模板
视频生成失败或卡在分段	显存不足、模型未下载、工作流损坏	用第 9.2 节运行核对；确认模型与磁盘资源
依赖安装缓慢或失败	网络或缓存问题	`uv cache clean` 后 `uv sync`；Docker 场景可用 `USE_CN_MIRROR=true` 加速
Docker 下找不到 `config.yaml`	未初始化或误将文件挂成目录	compose 中 init 容器会自动以示例文件重建（见 2.3 节）

附录 B 术语表

术语	英文	定义
分镜	Scene	视频的基本组成单元，一句或一段解说对应一个画面与配音
大语言模型	LLM (Large Language Model)	把主题扩写为结构化文案的文本模型
文本转语音	TTS (Text-to-Speech)	把解说文本合成为人声语音的技术
声音克隆	Voice Cloning	以上传的参考音频为音色样本，合成相近的人声，依赖支持克隆的 TTS 工作流
ComfyUI	—	基于节点连线方式的生成式 AI 图形编排环境，运行图像、视频与语音模型
工作流	Workflow	描述生成任务中模型、节点与参数的 JSON 文件，以 `image_` 、 `video_` 、 `tts_` 前缀区分用途
RunningHub	—	提供云端 ComfyUI 计算设备（24GB 与 48GB 显存机型）的托管平台
直连 API	Direct API	绕过 ComfyUI 直接调用模型供应商官方接口的途径
视频模板	Template	以 HTML 与 CSS 编写的单分镜画面版式文件，以 `static_` 、 `image_` 、 `video_` 前缀区分媒体依赖
提示词前缀	Prompt Prefix	拼接在每个画面描述前的英文风格词，用于统一整条视频的视觉风格
数字人	Digital Human	由 AI 生成的讲解人物形象，承担画面中的播报角色
图生视频	Image-to-Video	输入静态图像，延展为带运动的短动态片段
动作迁移	Motion Transfer	将参考视频中的动作应用到参考图角色上
自定义素材	Custom Materials	上传自有照片与视频，由 AI 分析素材并生成配套解说脚本的功能
BGM	Background Music	叠加在人声解说之下的背景音乐音轨
反推	Reverse Prompt	由已有图像反向生成提示词，用于参考定制的风格描述
uv	—	用于管理 Python 项目依赖与虚拟环境的工具
ffmpeg	—	音视频处理工具，负责成片的音频与视频合成
Streamlit	—	用于构建 Web 界面的 Python 框架，本项目 Web 前端基于它
ASGI / uvicorn	—	Python 异步 Web 服务协议及其常用服务器实现之一，承载 REST API
Swagger UI	—	浏览 HTTP 接口定义与在线试调的页面，本项目位于 8000 端口的 `/docs`